

SVEUČILIŠTE JOSIPA JURJA STROSSMAYERA U OSIJEKU
FAKULTET ELEKTROTEHNIKE, RAČUNARSTVA I
INFORMACIJSKIH TEHNOLOGIJA OSIJEK

Sveučilišni diplomski studij Računarstvo
Izborni blok Informacijske i podatkovne znanosti

PREDVIĐANJE POTROŠNJE ENERGIJE POMOĆU VREMENSKIH NIZOVA

Seminarski rad

Andrej Damašek

Osijek, 2026.

Sadržaj

1.	Uvod.	2
2.	Skup podataka i priprema podataka.	3
2.1.	Skup podataka	3
2.2.	Eksplorativna analiza podataka	3
2.3.	Predobrada podataka	8
3.	Opis korištenih metoda i algoritama.	9
3.1.	Metode poboljšavanja modela.	9
3.2.	Random Forest	9
3.3.	XGBoost	11
3.4.	LSTM.	12
3.5.	GRU	14
3.6.	TCN	15
3.7.	Transformer	17
4.	Metrike i usporedba rezultata.	19
4.1.	Metrike evaluacije.	19
4.2.	Usporedba rezultata.	19
5.	Zaključak.	22
	Prilog.	24

1. UVOD

Učinkovito upravljanje elektroenergetskim sustavima uvelike ovisi o mogućnosti pouzdanog predviđanja buduće potrošnje električne energije. Precizne prognoze potražnje omogućuju optimalno planiranje proizvodnih kapaciteta, smanjenje troškova rada sustava te povećanje sigurnosti i stabilnosti opskrbe električnom energijom. Razvojem pametnih mreža i rastućim uključivanjem obnovljivih izvora energije, koji su često podložni vremenskim utjecajima, predviđanje potrošnje postaje sve važnije područje istraživanja.

Problem koji se rješava u ovom radu odnosi se na predviđanje buduće potrošnje električne energije na temelju povijesnih podataka. Takvi podaci prirodno su organizirani u obliku vremenskih nizova, gdje su vrijednosti potrošnje zabilježene u određenim vremenskim intervalima. Analiza vremenskih nizova omogućuje prepoznavanje obrazaca, sezonalnosti i trendova koji se ponavljaju tijekom dana, tjedana ili godišnjih razdoblja, a koji mogu biti iskorišteni za izradu modela predviđanja.

Motivacija za rješavanje ovog problema proizlazi iz potrebe za optimizacijom energetske resursa i povećanjem učinkovitosti elektroenergetskih sustava. Precizna predviđanja omogućuju bolje upravljanje proizvodnim kapacitetima, smanjuju rizik od preopterećenja mreže te doprinose smanjenju nepotrebnih troškova. Osim toga, razvoj metoda strojnog i dubokog učenja otvorio je mogućnost primjene naprednih algoritama koji mogu uspješno modelirati složene obrasce prisutne u podacima o potrošnji energije.

Za potrebe ovog rada korišten je skup podataka koji sadrži podatke o desetominutnoj potrošnji električne energije s vremenskim značajkama. Prije izrade modela provedena je eksploratorna analiza podataka (EDA) korištenjem biblioteke Matplotlib kako bi se identificirali trendovi, sezonske promjene i potencijalne nepravilnosti u podacima.

Postoji više pristupa predviđanju vremenskih nizova. Tradicionalni pristupi temelje se na metodama strojnog učenja, poput Random Forest Regressor modela i algoritama temeljenih na gradijentnom pojačavanju stabala (XGBoost), koji koriste ručno definirane značajke za modeliranje odnosa između ulaznih podataka i budućih vrijednosti potrošnje. S druge strane, metode dubokog učenja, poput LSTM (engl. *Long Short-Term Memory*) i GRU (engl. *Gated Recurrent Unit*) mreža, posebno su prilagođene radu s vremenskim nizovima jer mogu učiti dugoročne ovisnosti između podataka. Noviji pristupi, poput TCN (engl. *Temporal Convolutional Network*) i Autoformer modela, koriste napredne arhitekture za učinkovito modeliranje vremenskih obrazaca i dugoročnih trendova.

U okviru ovog projektnog zadatka implementirano je i uspoređeno više modela strojnog i dubokog učenja za predviđanje potrošnje električne energije. Njihova uspješnost evaluirana je korištenjem metrika MAE (engl. *Mean Absolute Error*) i RMSE (engl. *Root Mean Squared Error*), dok su rezultati prikazani grafičkom usporedbom stvarnih i predviđenih vrijednosti potrošnje. Cilj rada je utvrditi koji pristup ostvaruje najveću točnost predviđanja na promatranom skupu podataka.

2. SKUP PODATAKA I PRIPREMA PODATAKA

2.1. Skup podataka

Za potrebe izrade sustava za predviđanje potrošnje električne energije korišten je skup podataka *Electric Power Consumption* dostupan na platformi Kaggle [1]. Skup podataka sadrži podatke prikupljene svakih 10 minuta u vremenskom rasponu od **1. siječnja 2017. do 30. prosinca 2017.** za grad Tétouan u Maroku te uključuje informacije o potrošnji električne energije i meteorološkim uvjetima opisanim u Tablici 2.1. Ukupno skup podataka sadrži **52 416 zapisa** i **9 stupaca**.

Tablica 2.1: Opis značajki skupa podataka

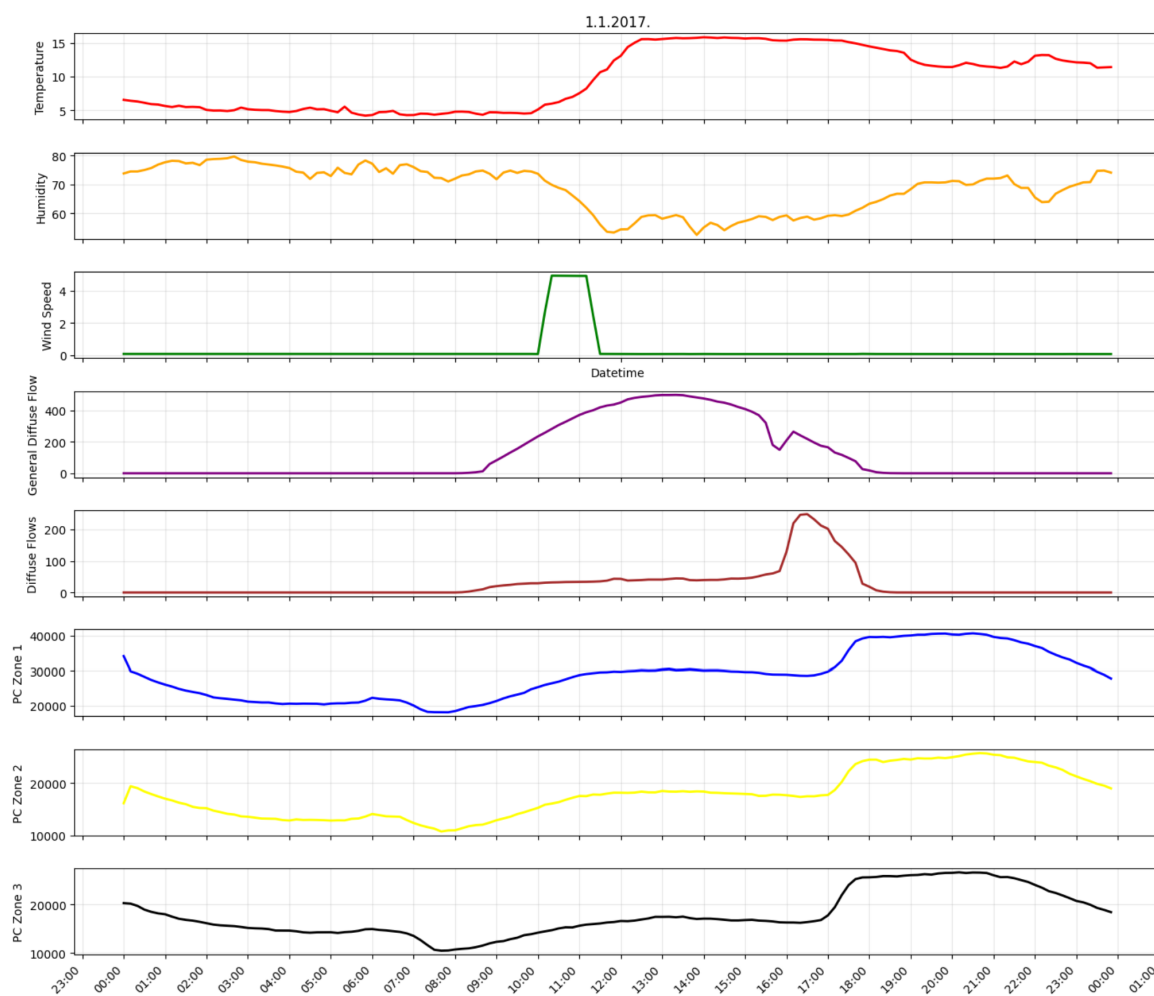
Naziv stupca	Tip	Opis
Datetime	Vremenska oznaka	Datum i vrijeme mjerenja (interval: 10 min)
Temperature	Numerički	Temperatura zraka u °C
Humidity	Numerički	Relativna vlažnost zraka (%)
WindSpeed	Numerički	Brzina vjetra (m/s)
GeneralDiffuseFlows	Numerički	Difuzno sunčevo zračenje – opće
DiffuseFlows	Numerički	Difuzno sunčevo zračenje
PowerConsumption_Zone1	Numerički	Potrošnja energije u zoni 1 (kW)
PowerConsumption_Zone2	Numerički	Potrošnja energije u zoni 2 (kW)
PowerConsumption_Zone3	Numerički	Potrošnja energije u zoni 3 (kW)

U ovom radu korišteni su podaci zone jedan (**PowerConsumption_Zone1**), dok su podaci za preostale dvije zone uklonjeni kako bi se pojednostavio proces modeliranja. Budući da su podaci izvorno zabilježeni svakih 10 minuta, agregirani su na satnu razinu, čime je dobiveno ukupno **8736 satnih zapisa** koji su korišteni za treniranje i evaluaciju modela.

2.2. Eksplorativna analiza podataka

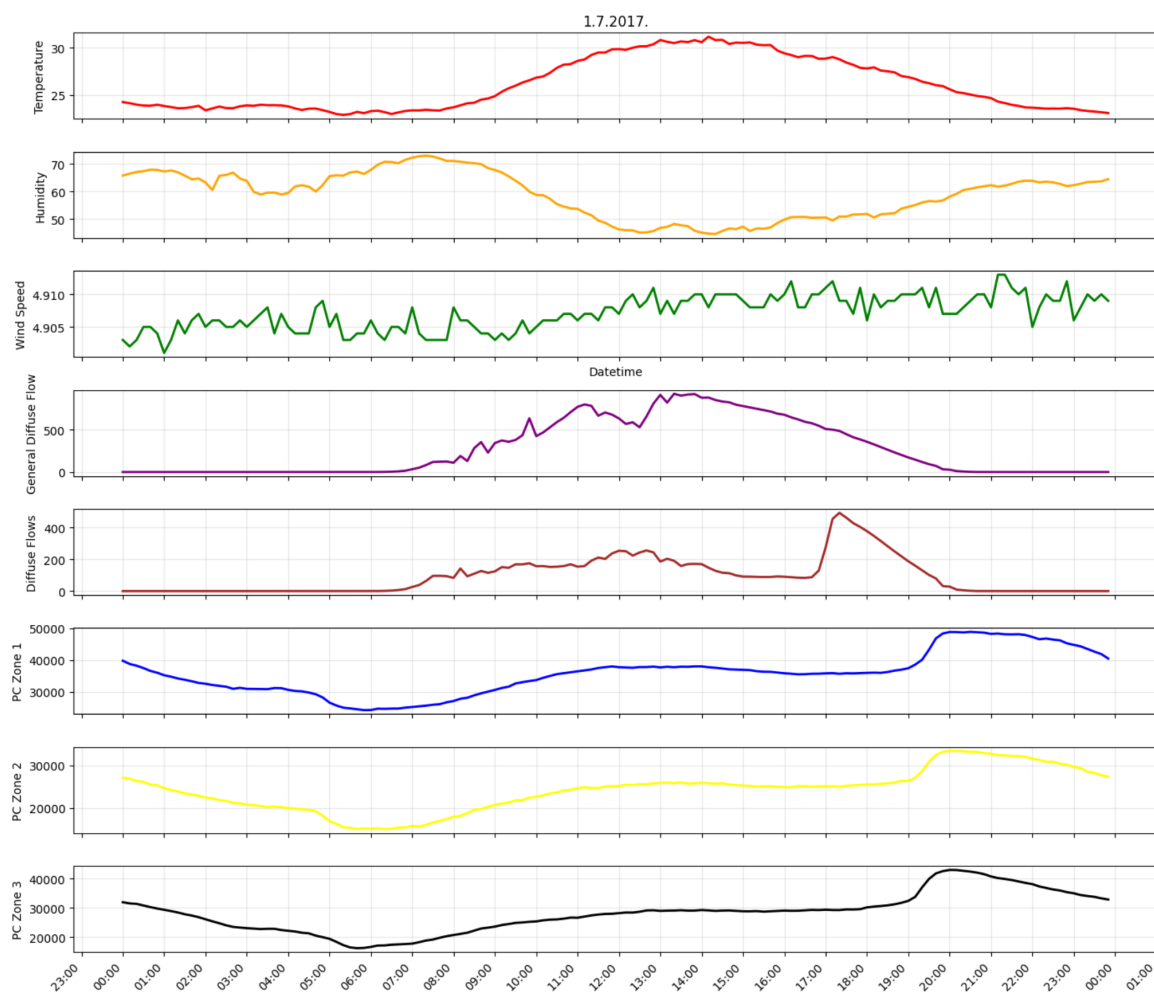
Prije izrade modela provedena je eksplorativna analiza podataka (EDA) s ciljem boljeg razumijevanja karakteristika skupa podataka i identifikacije mogućih obrazaca u potrošnji električne energije. Analiza obuhvaća vizualizaciju dnevnih i sezonskih profila potrošnje, pregled statističkih distribucija meteoroloških varijabli te ispitivanje korelacijskih odnosa između značajki.

Analiza je provedena korištenjem biblioteke Matplotlib. Prikazane su vrijednosti potrošnje energije i meteoroloških varijabli tijekom jednog dana kako bi se uočile dnevne oscilacije, kao što je prikazano na slici 2.1.



SL. 2.1: *Prikaz svih mjerenih varijabli za prvi dan u skupu podataka*

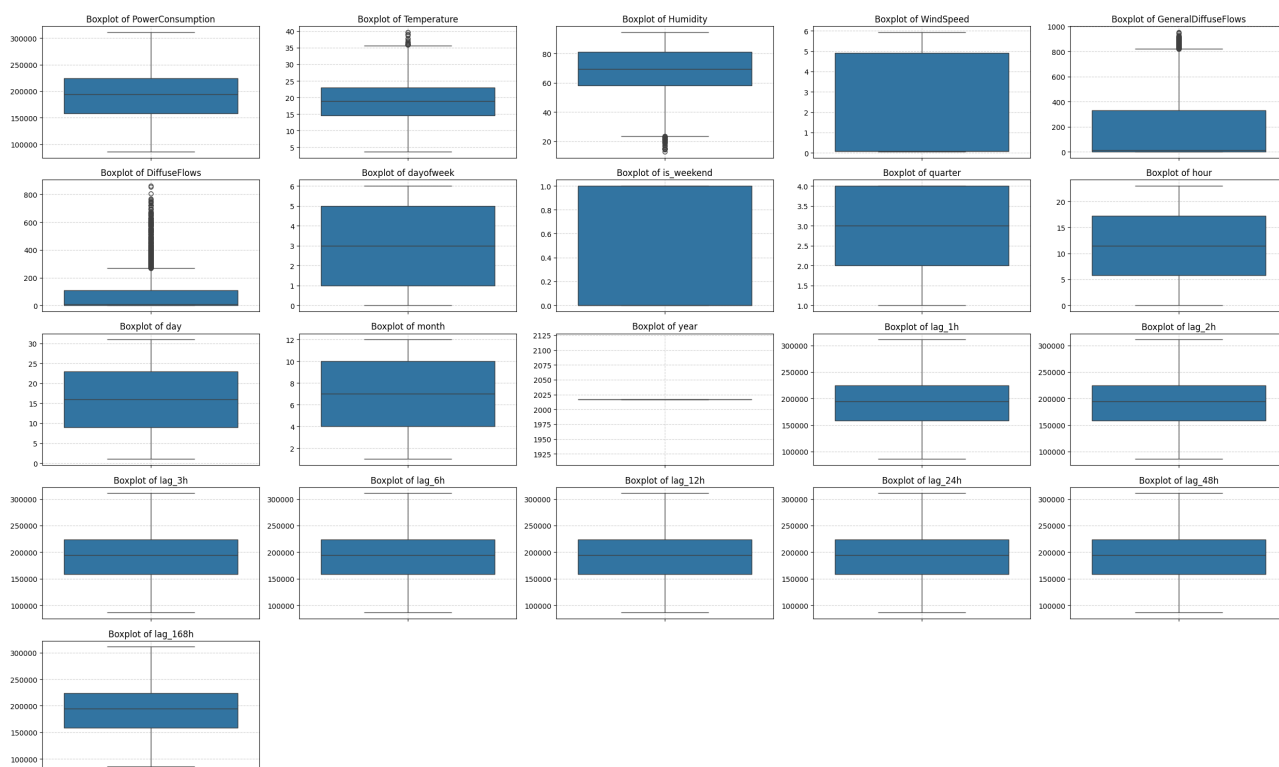
Dodatno je analiziran prvi dan ljetnog razdoblja (srpanj) kako bi se usporedilo ponašanje sustava u različitim vremenskim uvjetima, kao što je prikazano na slici 2.2.



SL. 2.2: *Prikaz svih mjerenih varijabli za prvi dan u srpnju*

Vizualizacijom dnevnih profila uočen je izražen dnevni ritam potrošnje: potrošnja pada na minimum između 5:00 i 8:00 sati ujutro, a doseže maksimum u večernjim satima između 17:00 i 22:00 sata. Također je utvrđeno da razlike između zona PowerConsumption_Zone1, PowerConsumption_Zone2 i PowerConsumption_Zone3 nisu značajne odnosno da prate iznimno sličan vremenski trend, stoga je za modeliranje odabrana samo zona 1 (PowerConsumption_Zone1) radi smanjenja dimenzionalnosti ulaznog prostora i sprječavanja pretreniranja (engl. *overfitting*).

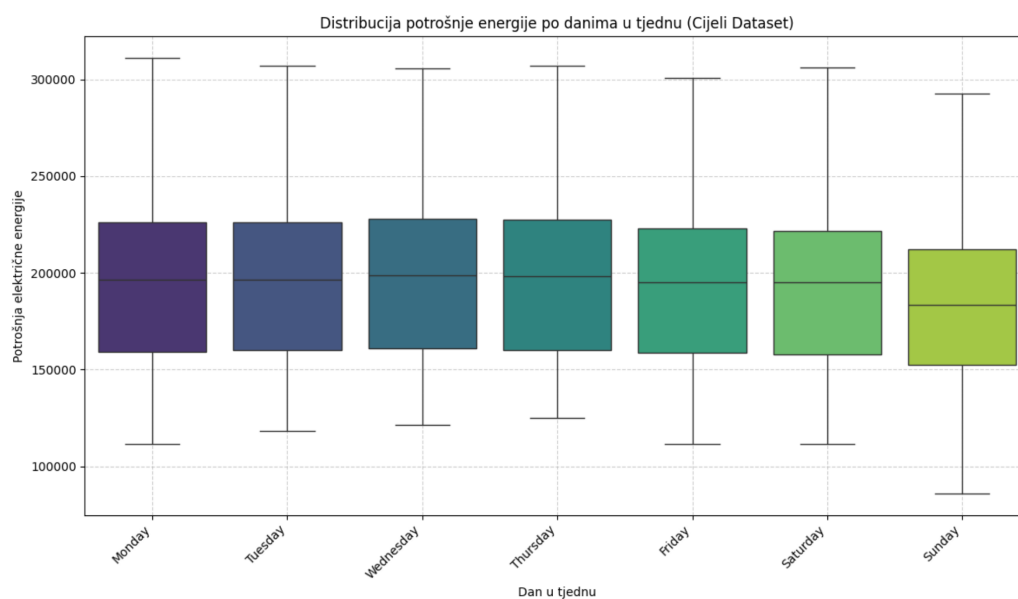
Na slici 2.3 vizualizirani su rasponi, medijani te prisutnost stršećih vrijednosti (engl. *outliers*) za sve numeričke značajke, što omogućuje dublji uvid u distribuciju podataka prije procesa treniranja.



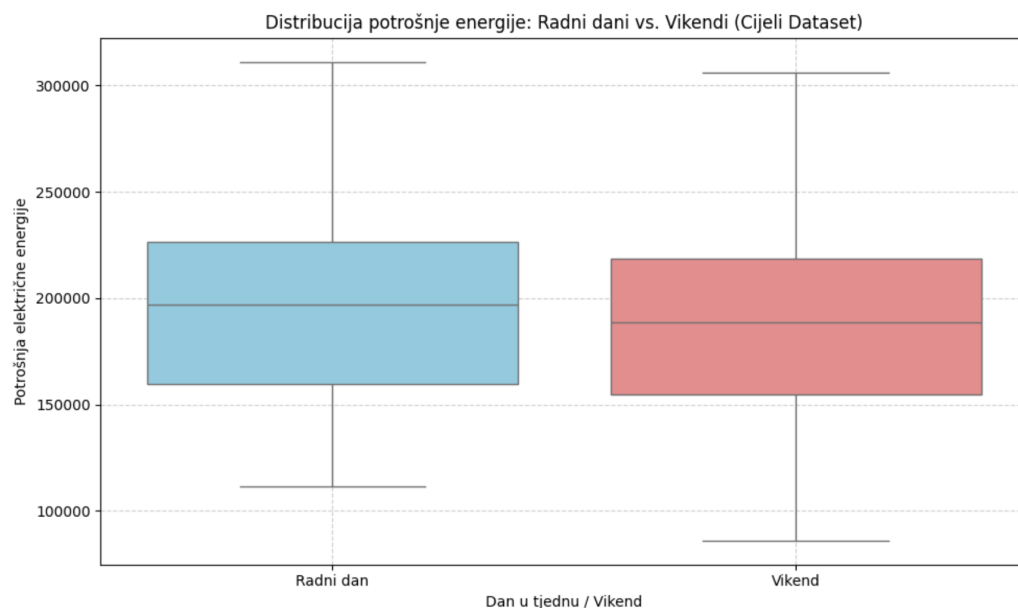
SL. 2.3: *Boxplotovi svih varijabli u skupu podataka*

Detaljnou analizom meteoroloških varijabli vidljivi su rasponi temperature od 3,2 °C do 40,0 °C s medijanom od 18,8 °C, što ukazuje na izraženu sezonsku varijabilnost karakterističnu za mediteransku klimu grada Tétouana. Relativna vlažnost zraka kreće se u rasponu između 11,3% i 94,2%, s medijanom iznad 60%, što upućuje na pretežito vlažne atmosferske uvjete tijekom godine. Brzina vjetra bilježi stabilnije vrijednosti u rasponu od 0,05 m/s do 6,48 m/s.

Na slici 2.4 prikazana je distribucija potrošnje energije po danima u tjednu iz koje se može vidjeti da je u prosjeku potrošnja od ponedjeljka do subote jednaka, a nedjeljom je značajno manja. A slika 2.5 potvrđuje da je u prosjeku vikendom manja potrošnja nego radnim danom.

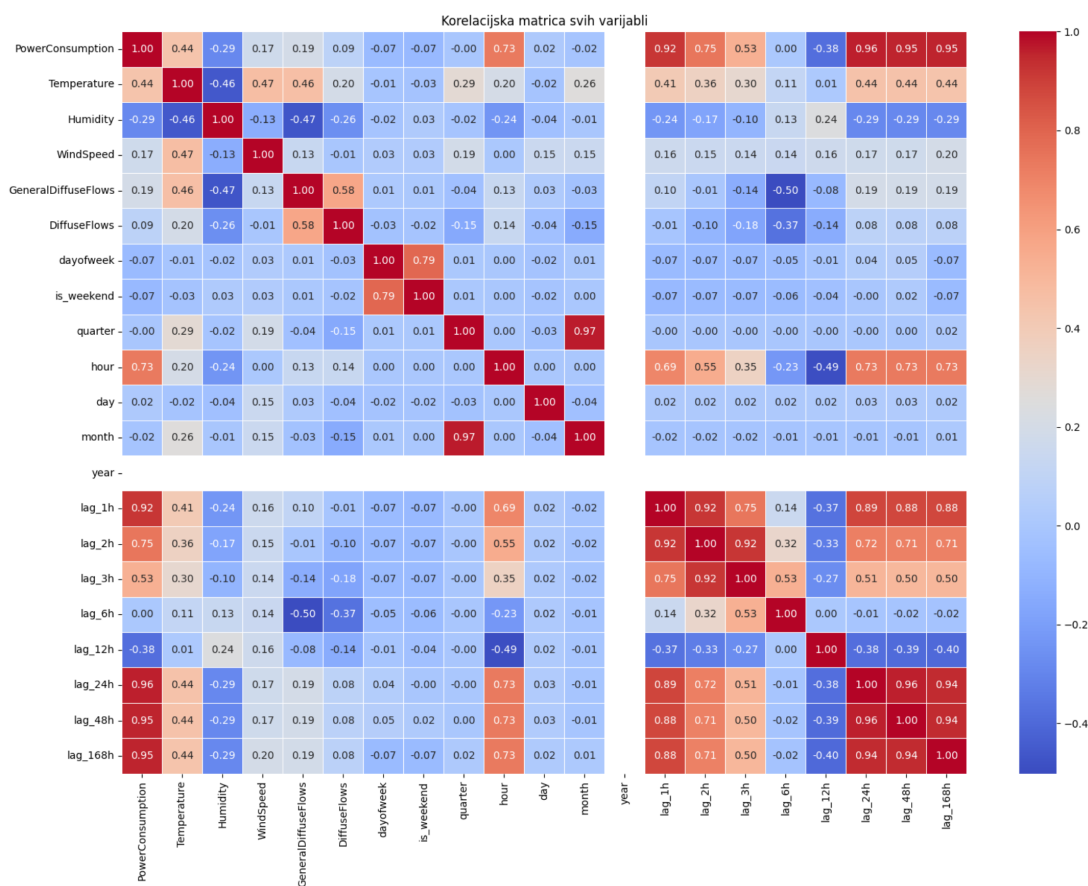


SL. 2.4: *Distribucija potrošnje po danima*



SL. 2.5: *Distribucija potrošnje za radni dan i vikend*

Analizom korelacijske matrice (Slika 2.6) utvrđeno je da između zaostale vrijednosti potrošnje (osobito `lag_24h` i `lag_168h`) i trenutne potrošnje postoji visoka pozitivna korelacija, što potvrđuje izraženu dnevnu i tjednu periodičnost u podacima. Također je utvrđeno da vrijednosti `lag_12h` i `Humidity` imaju izrazito negativnu korelaciju.



SL. 2.6: *Korelacijska matrica numeričkih značajki*

2.3. Predobrada podataka

Originalni skup podataka predobrađen je kroz nekoliko koraka:

1. **Odabir zone i resampling** – Za modeliranje su korišteni podaci o potrošnji električne energije u zoni 1 (`PowerConsumption_Zone1`). Izvorni podaci zabilježeni svakih 10 minuta agregirani su na satnu razinu zbrajanjem vrijednosti potrošnje unutar svakog sata, dok su meteorološke varijable agregirane računanjem srednje vrijednosti.
2. **Inženjerstvo vremenskih značajki** – Iz vremenske oznake izvučene su dodatne značajke: sat u danu, dan u tjednu, mjesec, kvartal te binarna oznaka vikenda (`is_weekend`).
3. **Zaostale značajke** – Dodane su zaostale vrijednosti potrošnje (`lag_1h`, `lag_2h`, `lag_3h`, `lag_6h`, `lag_12h`, `lag_24h`, `lag_48h` i `lag_168h`) kako bi modeli mogli koristiti informacije o prethodnoj potrošnji te prepoznati kratkoročne, dnevne i tjedne obrasce u vremenskom nizu.
4. **Podjela na trening i test skup** – Podaci su podijeljeni u omjeru 80%/20% za trening i testiranje.
5. **Normalizacija** – Za LSTM, TCN, GRU i Transformer modele provedena je normalizacija ulaznih podataka koristeći `MinMaxScaler`, čime su sve vrijednosti svedene u raspon $[0, 1]$ radi stabilnijeg i bržeg treniranja mreže.

3. OPIS KORIŠTENIH METODA I ALGORITAMA

3.1. Metode poboljšavanja modela

U cilju izgradnje što kvalitetnijih modela, uz predobradu opisanu u poglavlju 2.3, primijenjene su sljedeće dodatne tehnike:

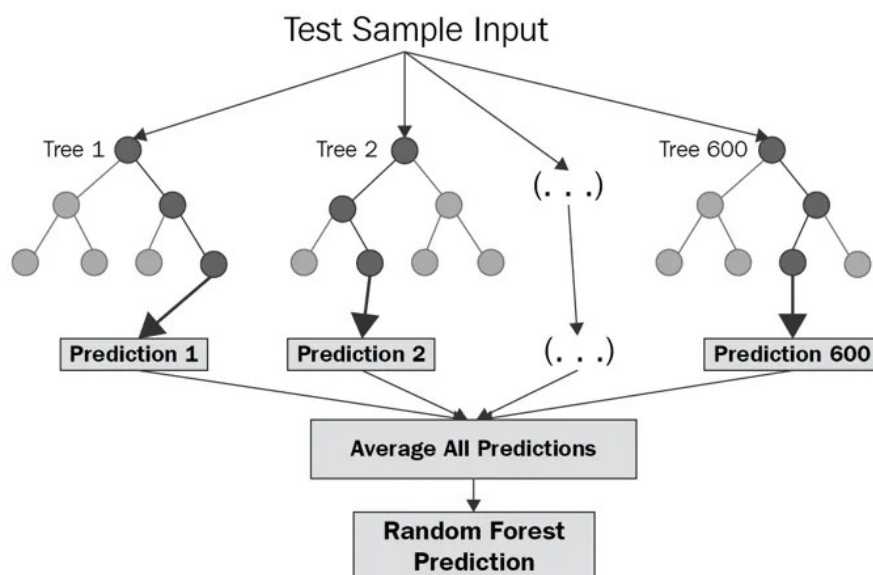
- **Rano zaustavljanje** (engl. *Early Stopping*) – svi modeli dubokog učenja koriste `EarlyStopping` s praćenjem validacijskog gubitka (`monitor='val_loss'`) i strpljivošću od 10 epoha (`patience=10`), čime se sprječava pretreniranje.
- **Dropout regularizacija** – u arhitekturama LSTM, TCN i GRU ugrađeni su Dropout slojevi koji nasumično isključuju dio neurona tijekom treniranja kako bi se smanjilo pretreniranje.
- **Podešavanje hiperparametara** (engl. *Hyperparameter Tuning*) – za modele dubokog učenja korišten je `KerasTuner` (algoritam Hyperband), a za Random Forest i XGBoost algoritam `GridSearchCV`.

3.2. Random Forest

Random forest [2] je ansambl algoritam strojnog učenja koji radi na principu kombiniranja većeg broja „slabih” modela (stabala odluke) kako bi se stvorio jedan „jak” model. Za regresijske probleme, predviđanje se dobiva usrednjavanjem izlaza pojedinih stabala. Svako pojedinačno stablo u random forestu trenira se na različitom, nasumično odabranom podskupu podataka (engl. *bagging*), a pri svakoj podjeli u stablu razmatra se samo slučajni podskup značajki. Na taj se način uvodi dodatna slučajnost i smanjuje međusobna sličnost stabala, što u konačnici smanjuje varijancu modela. Ova tehnika ujedno je robusna na prisutnost koreliranih značajki.

U kontekstu regresijskih, posebno nelinearnih problema, random forest se pokazuje vrlo učinkovitim jer stabla odluke implicitno dijele prostor značajki u mnoge manje regije te u svakoj od njih daju jednostavnu aproksimaciju ciljne funkcije. Prosjek velikog broja takvih stabala dovodi do glatke i fleksibilne aproksimacije složenih nelinearnih odnosa između ulaznih varijabli i ciljne varijable. U slučaju predviđanja potrošnje energije u vremenskim nizovima, random forest može naučiti obrasce kao što su ovisnost potrošnje o satu u danu, danu u tjednu, temperaturi, prethodnim vrijednostima potrošnje i njihovim međusobnim interakcijama, bez potrebe da unaprijed zadamo točan oblik funkcije koja te odnose opisuje.

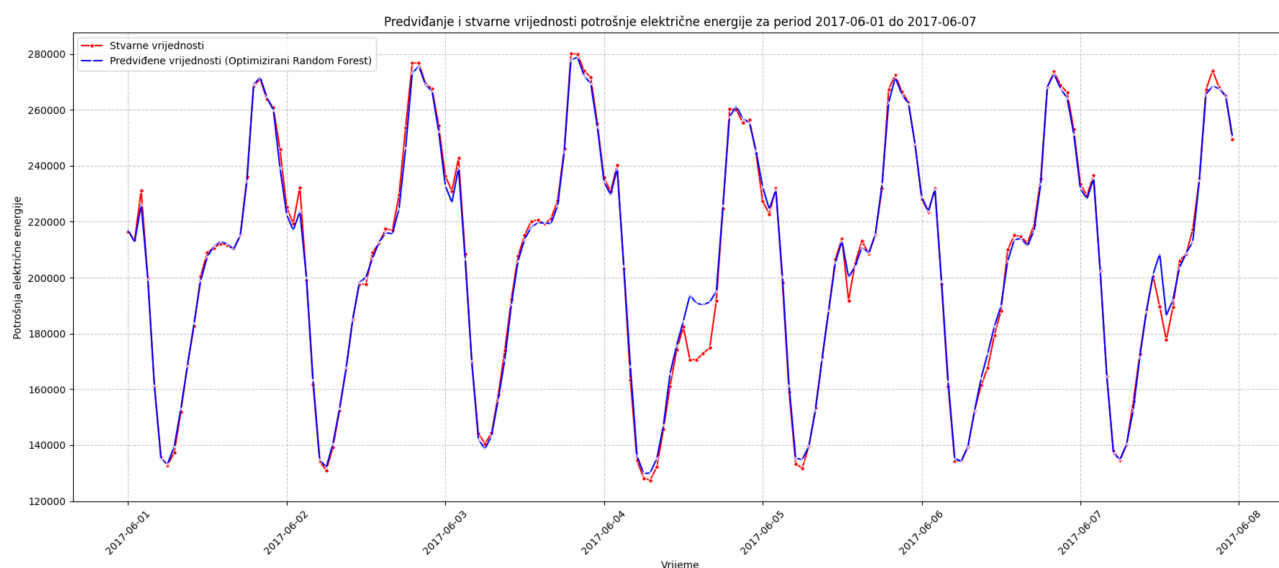
Na slici 3.7 prikazana je shematska arhitektura kako svako stablo u random forest algoritmu dolazi do svojega rješenja na temelju vlastite logike, dok se konačna predikcija dobiva usrednjavanjem pojedinačnih predviđanja svih stabala u šumi.



SL. 3.7: *Arhitektura Random Forest modela [3]*

U radu su implementirane tri varijante modela: osnovna varijanta (bez zaostalih vrijednosti), varijanta sa zaostalim vrijednostima te optimizirana varijanta s podešenim hiperparametrima (također sa zaostalim vrijednostima). Osnovna varijanta i varijanta sa zaostalim vrijednostima trenirane su s unaprijed zadanim vrijednostima hiperparametara (100 stabala odluke), dok su za optimiziranu varijantu hiperparametri određeni pomoću algoritma **GridSearchCV**. U sklopu postupka optimizacije podešeni su parametri broja stabala odluke i maksimalnog broja značajki koje se razmatraju pri svakoj podjeli.

Vizualizacija predviđanja optimiziranog Random Forest modela za period od 1. do 7. lipnja 2017. prikazana je na slici 3.8.

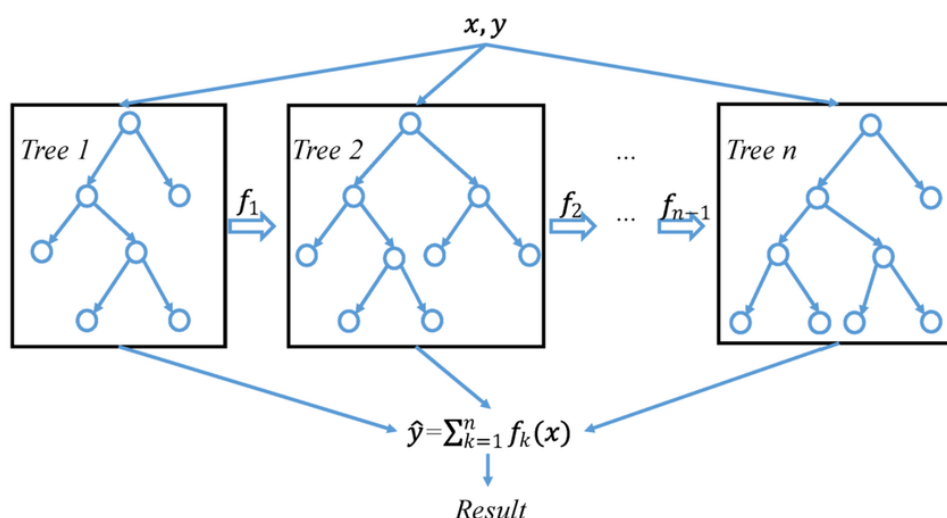


SL. 3.8: *Predviđene i stvarne vrijednosti potrošnje – optimizirani Random Forest*

3.3. XGBoost

XGBoost (engl. *Extreme Gradient Boosting*) [4] gradijentni je boosting algoritam koji iterativno gradi stabla odluke, pri čemu svako novo stablo ispravlja pogreške prethodnog minimiziranjem zadane funkcije gubitka. Za razliku od Random Foresta, gdje se stabla uče neovisno i njihove se predikcije usrednjavaju, XGBoost gradi stabla sekvencijalno tako da svako stablo predstavlja dodatnu korekciju postojeće aproksimacije, a cilj je minimizirati sumu gubitaka uz dodatak regularizacijskog člana koji kontrolira složenost modela. U praksi se model ciljne funkcije zapisuje kao zbroj predikcija više jednostavnih stabala, pri čemu svako stablo doprinosi manju korekciju, skaliranu faktorom učenja (engl. *learning rate*), što omogućuje stabilnije učenje i smanjenje rizika od pretreniranja.

Na slici 3.9 prikazana je shematska arhitektura XGBoost modela, u kojoj se vidi da se predikcija dobiva kao zbroj doprinosa više uzastopno izgrađenih stabala odluke, pri čemu svako novo stablo korigira pogreške koje su učinila prethodna stabla.



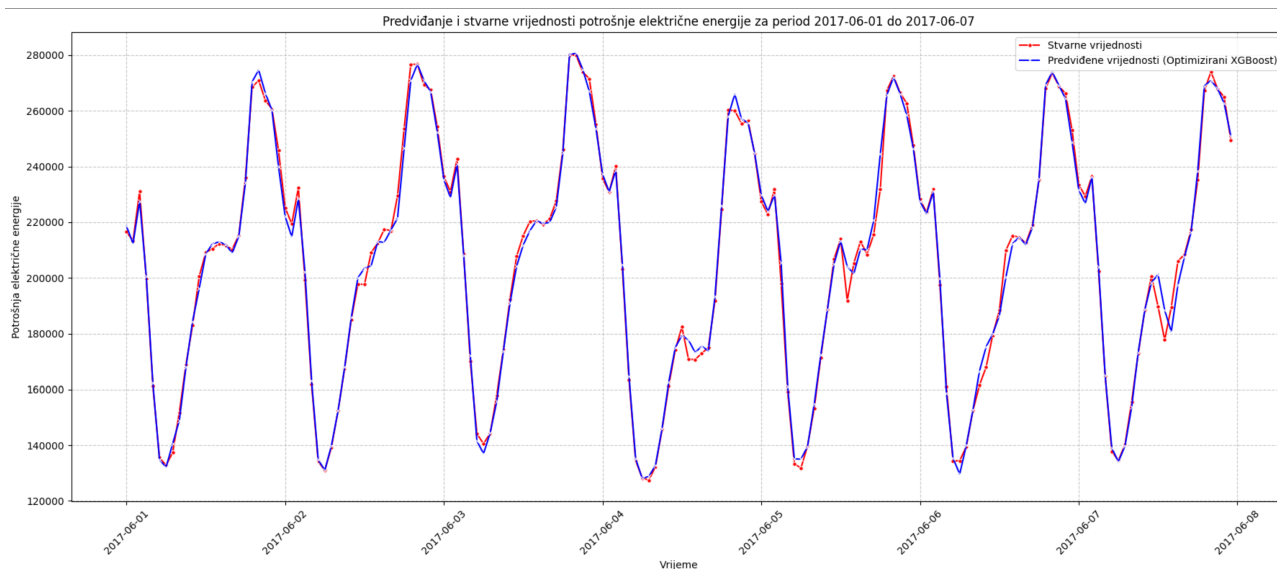
SL. 3.9: Arhitektura XGBoost modela [5]

Za razliku od klasičnih implementacija gradijentnog boostinga, XGBoost eksplicitno ugrađuje L1 i L2 regularizacijske parametre u ciljnu funkciju, čime se kažnjava prevelika složenost stabala (velik broj listova i velike vrijednosti težina u listovima) te dodatno smanjuje pretreniranje. Struktura pojedinog stabla slična je regresijskom stablu odluke: u svakom čvoru traži se podjela skupa primjera koja najviše smanjuje vrijednost ciljne funkcije, pri čemu XGBoost koristi aproksimaciju temeljenu na gradijentima i drugim derivatima funkcije gubitka, što ga čini računalno učinkovitim i skalabilnim na velike skupove podataka.

U radu su implementirane tri varijante XGBoost modela: osnovna varijanta bez zaostalih vrijednosti, varijanta sa zaostalim vrijednostima te optimizirana varijanta s podešenim hiperparametrima i uključenim zaostalim vrijednostima. Osnovne dvije varijante trenirane su s unaprijed zadanim vrijednostima hiperparametara, dok je za optimiziranu varijantu provedena pretraga hiperparametara pomoću algoritma **GridSearchCV**. Na taj su način istovremeno ispitane različite kombinacije broja boosting iteracija, maksimalne dubine stabala i faktora učenja, kako bi se pronašla konfiguracija koja daje najbolji kompromis između točnosti i robusnosti modela.

Optimizirana varijanta XGBoost modela sa zaostalim vrijednostima postigla je najbolje rezultate među svim testiranim modelima, što se posebno očituje u najnižim vrijednostima evaluacijskih metrike MAE i RMSE u usporedbi s ostalim pristupima. Vizualna usporedba

stvarnih i predviđenih vrijednosti potrošnje električne energije za razdoblje od 1. do 7. lipnja 2017. godine pokazuje vrlo dobro podudaranje, pri čemu XGBoost uspješno prati dnevne oscilacije potrošnje i hvata lokalne vršne vrijednosti, kako je prikazano na slici 3.10.

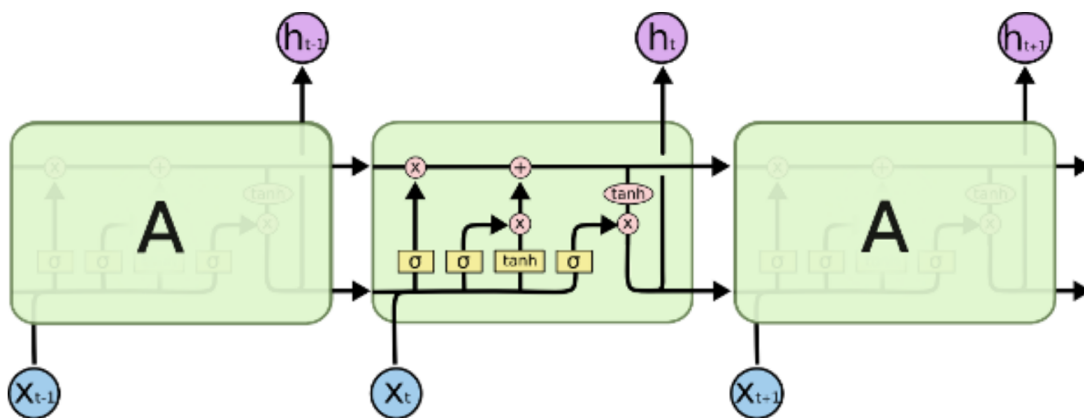


SL. 3.10: *Predviđene i stvarne vrijednosti potrošnje – optimizirani XGBoost*

3.4. LSTM

LSTM (engl. *Long Short-Term Memory*) [6] vrsta je rekurentne neuronske mreže posebno dizajnirane za modeliranje sekvencijalnih podataka i hvatanje dugoročnih ovisnosti koje klasične RNN (engl. *Recurrent Neural Networks*) arhitekture teško uče zbog problema nestajućih i eksplodirajućih gradijenata. LSTM uvodi tzv. memorijske ćelije koje u svakoj vremenskoj točki održavaju unutarnje stanje te koriste tri kapije – ulaznu, zaboravnu i izlaznu – kako bi kontrolirale koliko se nove informacije upisuju, koliko se prethodne zaboravljaju i koliki se dio unutarnjeg stanja prosljeđuje dalje prema izlazu mreže. Zahvaljujući takvoj strukturi, LSTM može „pamtiti” informacije kroz više vremenskih koraka i tako modelirati obrasce koji se protežu kroz dulje razdoblje, što ga čini posebno pogodnim za vremenske nizove, prirodni jezik i druge sekvencijalne podatke.

Na slici 3.11 prikazana je shematska arhitektura LSTM ćelije i pripadne mreže, gdje se može vidjeti kako se u svakom vremenskom koraku kombiniraju ulazni podaci, prethodno skriveno stanje i unutarnje stanje memorijske ćelije. Ulazna kapija odlučuje u kojoj se mjeri nove informacije upisuju u memorijsku ćeliju, zaboravna kapija regulira koje se informacije iz prošlosti brišu, dok izlazna kapija kontrolira koliki se dio ažuriranog stanja prenosi prema izlazu i sljedećem vremenskom koraku. Takva kombinacija kapija omogućuje stabilniji tok gradijenata kroz vrijeme te sprječava da korisne informacije prebrzo nestanu, što je ključno za precizno modeliranje potrošnje energije koja pokazuje sezonske i dnevne obrasce.

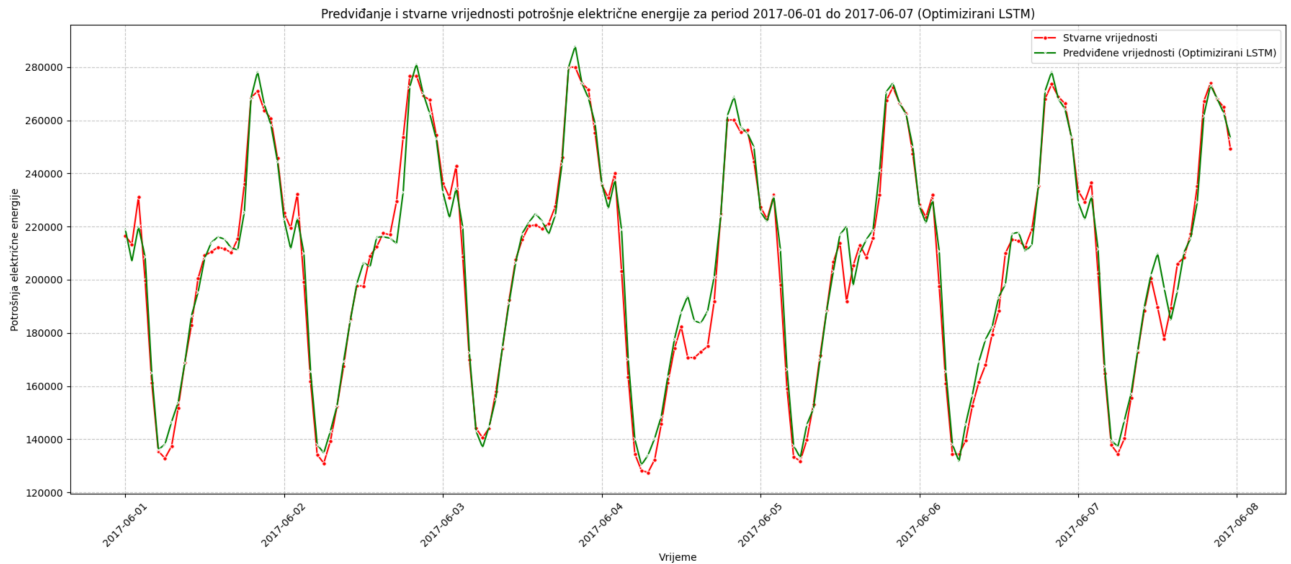


SL. 3.11: Arhitektura LSTM modela [7]

U radu je početna arhitektura LSTM modela sa zaostalim vrijednostima definirana kao dvostruki LSTM model s dropout regularizacijom. Prvi sloj je LSTM sloj sa 64 jedinice i opcijom `return_sequences=True`, što znači da taj sloj vraća cjelokupnu sekvencu skrivenih stanja i tako omogućuje da drugi LSTM sloj obrađuje informacije iz svih vremenskih koraka. Nakon toga slijedi Dropout sloj sa stopom od 20%, koji nasumično isključuje dio neurona tijekom treniranja i time smanjuje rizik od pretreniranja. Zatim je dodan drugi LSTM sloj s 32 jedinice, koji sažima informacije iz prethodnog sloja u kompaktniju reprezentaciju, te na kraju izlazni gusto povezani (*Dense*) sloj s jednom jedinicom, koji generira skalarno predviđanje potrošnje za zadani vremenski korak.

Model je kompajliran s Adam optimizatorom i MSE funkcijom gubitka, što znači da je tijekom učenja cilj bio minimizirati srednju kvadratnu pogrešku između stvarnih i predviđenih vrijednosti potrošnje. Treniranje je provedeno s veličinom paketa `batch_size=32` i validacijskim udjelom od 10%, čime se dio podataka izdvojio za praćenje generalizacijske sposobnosti modela tijekom učenja. Za optimiziranu varijantu LSTM modela hiperparametri poput broja jedinica u LSTM slojevima, dropout stope i stope učenja određeni su pomoću alata **KerasTuner Hyperband**, koji automatski pretražuje prostor hiperparametara i odabire kombinacije koje daju najbolje performanse prema zadanoj metričkoj funkciji.

Vizualizacija predviđanja optimiziranog LSTM modela za razdoblje od 1. do 7. lipnja 2017. godine prikazana je na slici 3.12.

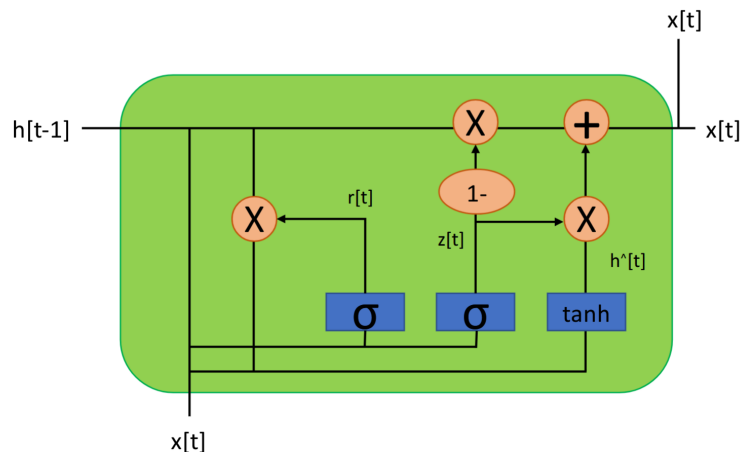


SL. 3.12: *Predviđanje i stvarne vrijednosti potrošnje – Optimizirani LSTM*

3.5. GRU

GRU (engl. *Gated Recurrent Unit*) [8] pojednostavljena je varijanta rekurentne neuronske mreže slična LSTM-u, ali s manjim brojem komponenti. Umjesto zasebnih ulaznih, izlaznih i zaboravnih kapija, GRU koristi dvije kapije: kapiju ažuriranja (engl. *update gate*) i kapiju poništavanja (engl. *reset gate*), koje zajednički kontroliraju koliko se informacija iz prošlih stanja zadržava, a koliko se novih informacija uvodi u trenutno skriveno stanje. Kapija ažuriranja određuje omjer između starog i novog stanja, dok kapija poništavanja regulira koliki utjecaj ima prethodno stanje pri izračunu kandidatskog stanja; na taj način GRU može učinkovito modelirati i kratkoročne i dugoročne ovisnosti u vremenskom nizu uz jednostavniju strukturu od LSTM-a. Zbog toga se u mnogim zadacima vremenskih nizova GRU pokazuje usporedivim s LSTM-om po točnosti, ali uz manji računalni trošak i kraće vrijeme treniranja.

Na slici 3.13 prikazana je arhitektura GRU ćelije, u kojoj se vidi kako kapija ažuriranja i kapija poništavanja zajedno upravljaju protokom informacija između prethodnog i trenutnog skrivenog stanja.

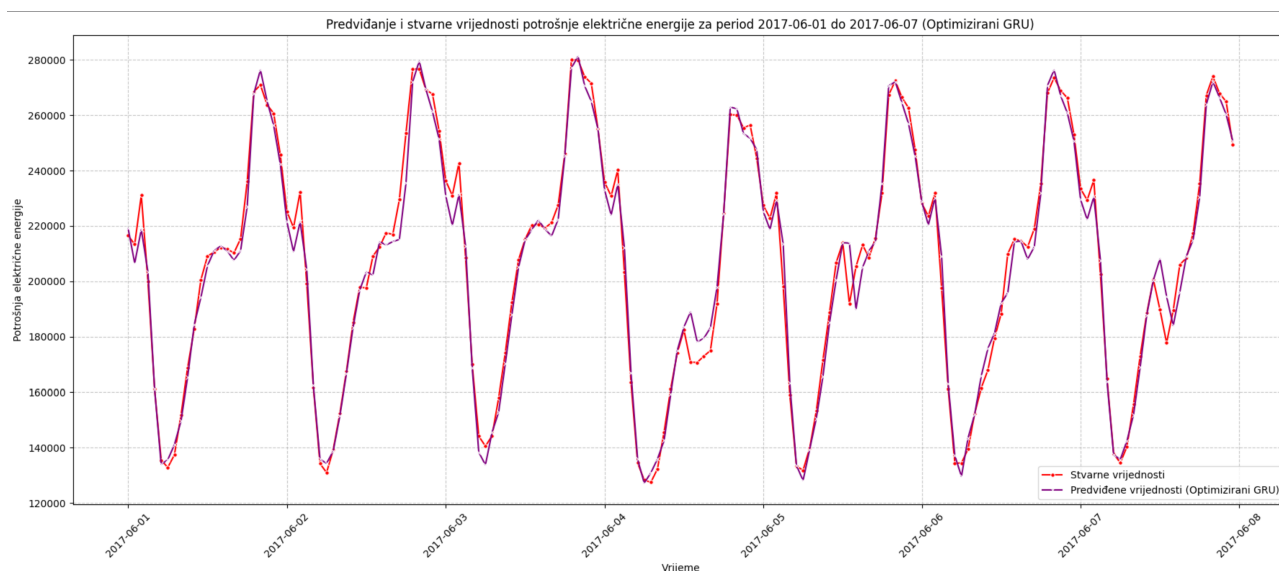


SL. 3.13: *Arhitektura GRU modela [9]*

Osnovna GRU arhitektura u ovom radu koristi zaostale vrijednosti potrošnje kao ulaz. Model se sastoji od jednog GRU sloja s 50 jedinica i nelinearnom aktivacijom 'relu', nakon čega slijedi izlazni gusto povezani (Dense) sloj s jednom jedinicom, koji generira predviđenu vrijednost potrošnje za zadani vremenski korak. Model je kompajliran s Adam optimizatorom i MSE funkcijom gubitka, čime je tijekom učenja cilj bio minimizirati srednju kvadratnu pogrešku između stvarnih i predviđenih vrijednosti potrošnje. Takva jednostavna arhitektura omogućuje relativno brzu obuku, što je pogodno kada se uspoređuje više različitih modela vremenskih nizova.

Optimizirana varijanta GRU modela dobivena je pretragom hiperparametara pomoću alata **KerasTuner**. U sklopu te pretrage ispitivane su različite kombinacije broja jedinica u GRU sloju, Dropout stope i stope učenja. U usporedbi s osnovnom arhitekturom, optimizirani model povećava kapacitet GRU sloja (više jedinica) i uklanja Dropout, ali istovremeno koristi relativno malu stopu učenja, što omogućuje stabilnije konvergiranje i bolju aproksimaciju složenih obrazaca u podacima.

Vizualizacija predviđanja optimiziranog GRU modela za razdoblje od 1. do 7. lipnja 2017. godine prikazana je na slici 3.14, gdje se može vidjeti koliko dobro model prati stvarne vrijednosti potrošnje kroz vrijeme.

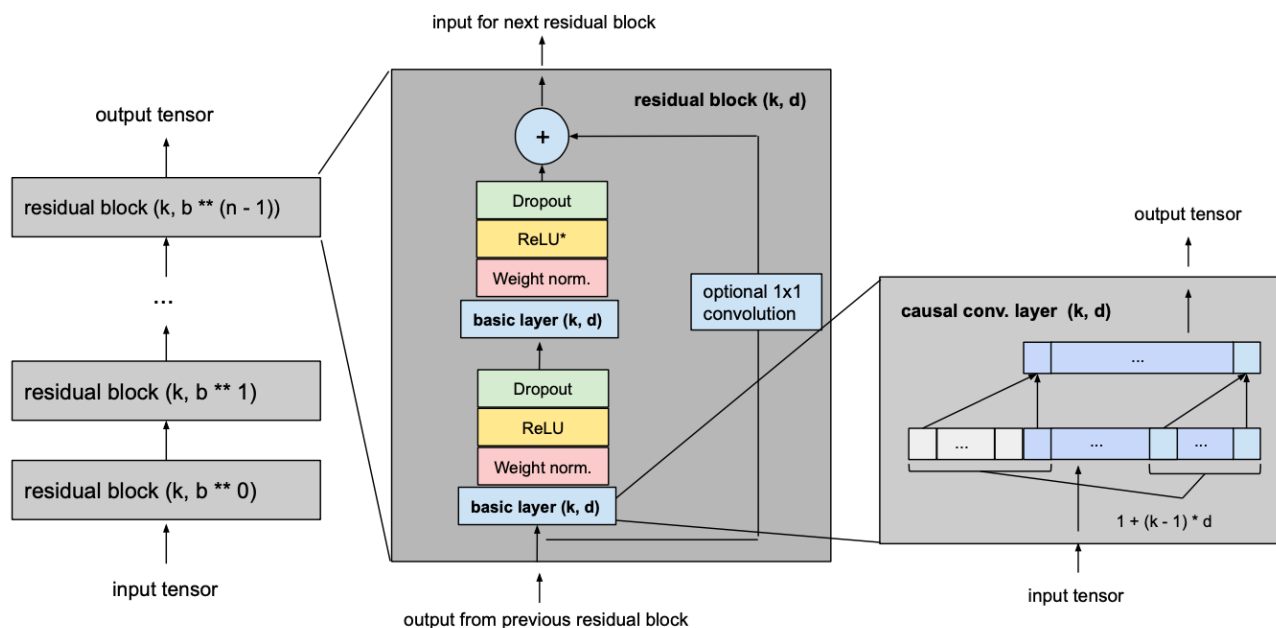


SL. 3.14: *Predviđanje i stvarne vrijednosti potrošnje – Optimizirani GRU*

3.6. TCN

TCN (engl. *Temporal Convolutional Network*) [10] arhitektura je neuronske mreže namijenjena modeliranju vremenskih nizova koja se temelji na kauzalnim dilatacijskim konvolucijama. Za razliku od rekurentnih mreža, TCN obrađuje cijeli ulazni slijed paralelno, što omogućuje brže treniranje i učinkovito iskorištavanje suvremenih grafičkih procesora. Kauzalno punjenje (engl. *causal padding*) pritom osigurava da model pri predviđanju koristeći vrijednost u vremenu t ne vidi buduće vrijednosti, odnosno da se izlaz u trenutku t računa isključivo na temelju trenutnih i prošlih uzoraka, što je ključno za ispravno modeliranje vremenskih nizova.

Na slici 3.15 prikazana je arhitektura TCN modela, pri čemu je duljina ulaznog prozora označena s l (jednaka `input_length`), veličina jezgre s k (jednaka `kernel_size`), a baza dilatacije s b , uz uvjet $k \geq b$ i minimalan broj rezidualnih blokova n potreban za pokrivanje cijele povijesti signala, koji se može izračunati na temelju tih parametara.

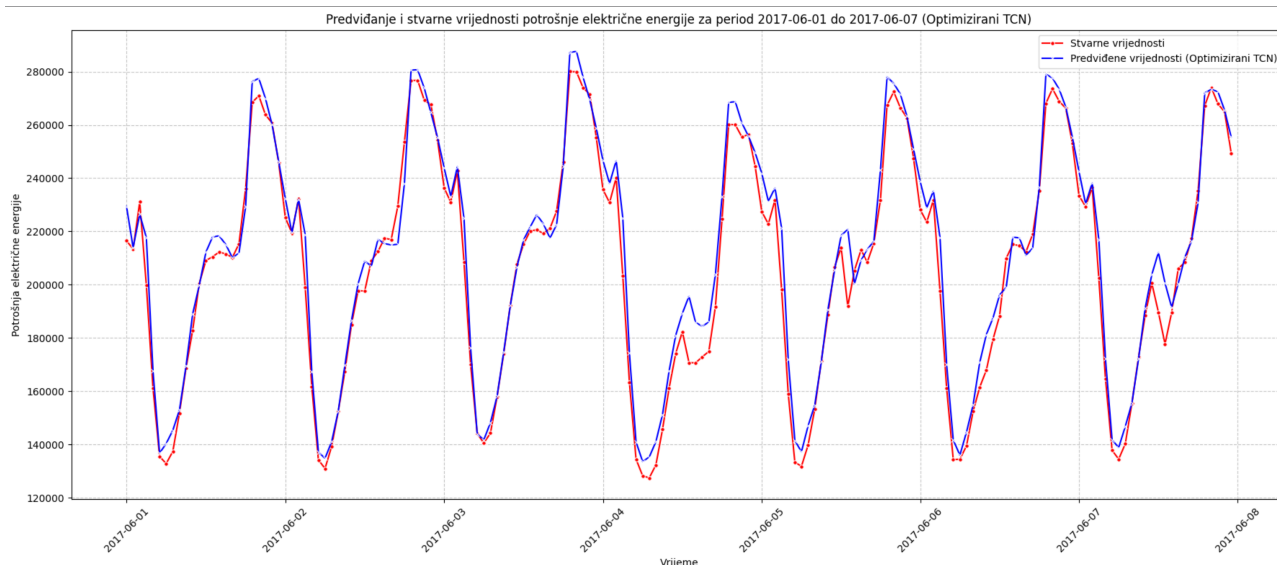


SL. 3.15: *Arhitektura TCN modela*[11]

Osnovna TCN arhitektura u ovom radu koristi zaostale vrijednosti potrošnje kao ulaz te se sastoji od konvolucijskih slojeva s kauzalnim i dilatacijskim filtrima. U početnoj konfiguraciji primijenjeno je 64 filtra (`nb_filters=64`) s veličinom jezgre 2 (`kernel_size=2`), jedan skup rezidualnih blokova (`nb_stacks=1`), dilatacije, kauzalno punjenje te Dropout stopa od 20%. Rezidualni blokovi omogućuju da se ulazni signal zbraja s izlazom konvolucijskih slojeva, čime se stabilizira treniranje dubljih mreža i olakšava propagacija gradijenata kroz model, dok dilatacije povećavaju efektivni receptivni opseg bez potrebe za velikim brojem slojeva.

Optimizirana varijanta TCN modela određena je pretragom hiperparametara pomoću alata `KerasTuner`, pri čemu su pretraživane kombinacije broja filtara, veličine jezgre, broja rezidualnih blokova, Dropout stope i stope učenja. U odnosu na osnovnu arhitekturu, optimizirani model koristi dublju strukturu s dva skupa rezidualnih blokova i bez Dropout regularizacije, ali uz pažljivo podešenu stopu učenja, što omogućuje preciznije hvatanje obrazaca u vremenskom nizu uz zadržavanje dobre generalizacije.

Vizualizacija predviđanja optimiziranog TCN modela za razdoblje od 1. do 7. lipnja 2017. godine prikazana je na slici 3.16.

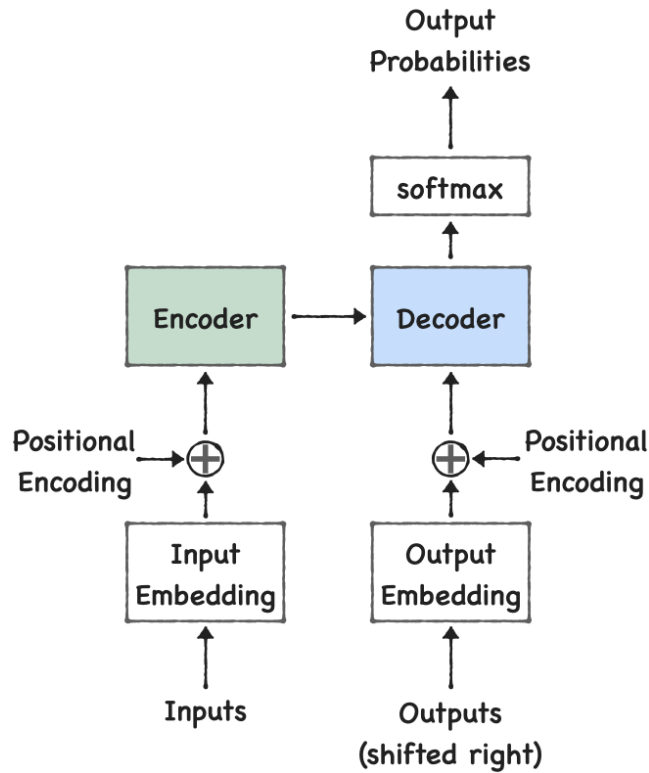


SL. 3.16: *Predviđanje i stvarne vrijednosti potrošnje – Optimizirani TCN*

3.7. Transformer

Transformer arhitektura [12] originalno je razvijena za zadatke obrade prirodnog jezika, ali se pokazala učinkovitom i za predviđanje vremenskih nizova. Temelji se na mehanizmu višeglavne samopažnje (engl. *multi-head self-attention*) koji modelu omogućuje da istovremeno prati ovisnosti između svih vremenskih koraka u ulaznom slijedu, neovisno o njihovoj međusobnoj udaljenosti. U takvom modelu svaka vremenska točka reprezentirana je vektorom značajki dimenzije d_{model} , dok mehanizam samopažnje kroz više glava uči koje dijelove ulaza treba naglasiti pri izračunu konačne reprezentacije za pojedini vremenski korak. Zahvaljujući paralelnoj obradi cijelog slijeda i mogućnosti učenja dugoročnih ovisnosti, transformer se prirodno prilagođava zadacima regresije nad vremenskim nizovima, gdje izlaz predstavlja kontinuiranu vrijednost, poput buduće potrošnje energije.

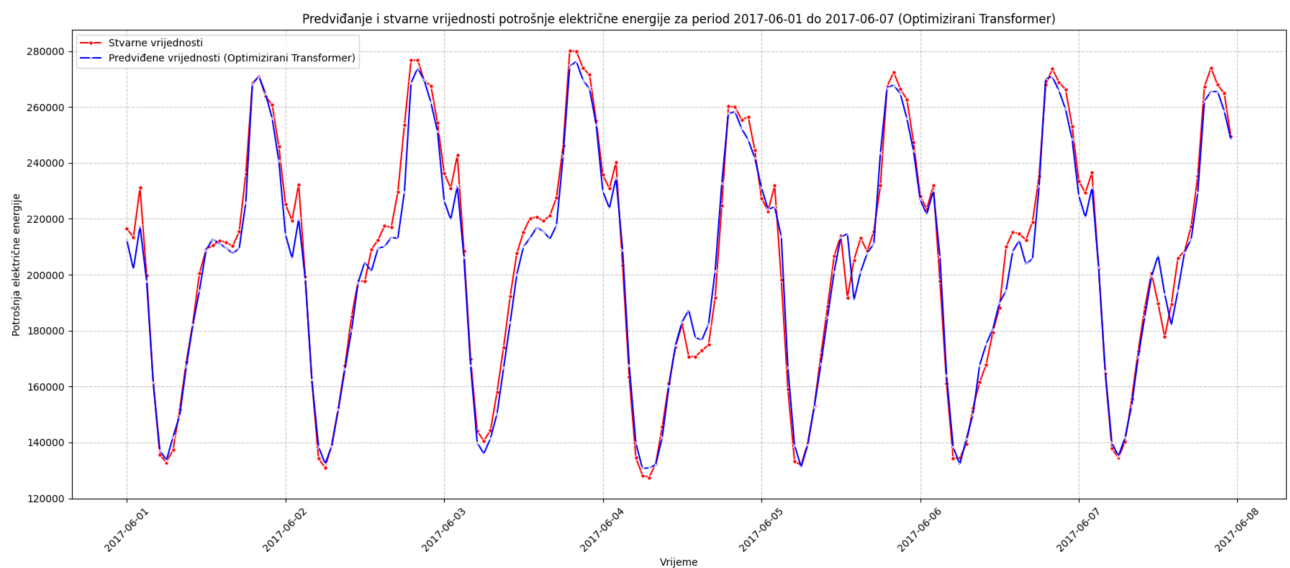
Na slici 3.17 prikazana je opća arhitektura Transformer modela za vremenske nizove, temeljena na encoder–decoder strukturi s pozicijskim kodiranjem ulazne i izlazne sekvence. U ovom radu korišten je encoder dio takve arhitekture prilagođen regresijskom predviđanju potrošnje električne energije.



SL. 3.17: Arhitektura Transformer modela [13]

Implementirana Transformer arhitektura sa zaostalim vrijednostima koristi dimenziju ugrađivanja `embed_dim` jednaku broju značajki (20), `num_heads` = 4 glave pažnje, veličinu potpuno povezane mreže `ff_dim` = 32 te Adam optimizator kao algoritam učenja. Optimizirana varijanta određena je pretragom hiperparametara pomoću alata `KerasTuner`.

Vizualizacija predviđanja optimiziranog Transformer modela za razdoblje od 1. do 7. lipnja 2017. godine prikazana je na slici 3.18, gdje se može vidjeti koliko dobro model prati stvarne vrijednosti potrošnje kroz vrijeme.



SL. 3.18: Predviđanje i stvarne vrijednosti potrošnje – Optimizirani Transformer

4. METRIKE I USPOREDBA REZULTATA

4.1. Metrike evaluacije

Za evaluaciju performansi izgrađenih modela korištene su dvije standardne regresijske metrike:

- **MAE** (engl. *Mean Absolute Error*) – srednja apsolutna pogreška, definirana kao:

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (4-1)$$

gdje je y_i stvarna vrijednost, $\hat{y}_i$ predviđena vrijednost, a n broj uzoraka. MAE je intuitivno interpretabilan jer izražava prosječnu pogrešku u istim jedinicama kao ciljna varijabla (kW).

- **RMSE** (engl. *Root Mean Squared Error*) – korijen srednje kvadratne pogreške, definiran kao:

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \quad (4-2)$$

RMSE penalizira veće pogreške jače od MAE-a, što ga čini posebno korisnim za identifikaciju modela koji proizvode izrazito loša predviđanja u pojedinim slučajevima.

4.2. Usporedba rezultata

Rezultati evaluacije svih modela na testnom skupu koji čini 20% ukupnog skupa podataka (1747 zapisa) prikazani su u Tablici 4.2.

Tablica 4.2: *Usporedba performansi svih modela na testnom skupu*

Tip modela	Konfiguracija	MAE	RMSE
Random Forest	Bez zaostalih vrijednosti	5590.03	8096.94
	Sa zaostalim vrijednostima	3748.59	5675.74
	Podešeni hiperparametri sa zaostalim vrijednostima	3701.32	5490.52
XGBoost	Bez zaostalih vrijednosti	5023.68	7042.05
	Sa zaostalim vrijednostima	3663.40	5288.39
	Podešeni hiperparametri sa zaostalim vrijednostima	3405.16	4879.57
LSTM	Bez zaostalih vrijednosti	34816.67	41985.28
	Sa zaostalim vrijednostima	4050.38	5690.72
	Podešeni hiperparametri sa zaostalim vrijednostima	3887.36	5380.85
GRU	Bez zaostalih vrijednosti	36279.72	44177.99
	Sa zaostalim vrijednostima	3783.78	5380.66
	Podešeni hiperparametri sa zaostalim vrijednostima	3680.79	5177.93
TCN	Bez zaostalih vrijednosti	35551.53	43067.13
	Sa zaostalim vrijednostima	10402.08	13549.63
	Podešeni hiperparametri sa zaostalim vrijednostima	4562.16	6165.11
Transformer	Sa zaostalim vrijednostima	4985.33	6802.16
	Podešeni hiperparametri sa zaostalim vrijednostima	3905.05	5397.74

Kako bi se dodatno kvantificirao utjecaj zaostalih vrijednosti i optimizacije hiperparametara, u tablici 4.3 prikazane su prosječne vrijednosti MAE i RMSE po konfiguraciji preko svih modela.

Tablica 4.3: *Prosječne vrijednosti MAE i RMSE po konfiguraciji modela*

Konfiguracija	Prosječni MAE	Prosječni RMSE
Bez zaostalih vrijednosti	24 652.33	28 673.88
Sa zaostalim vrijednostima	4 772.26	6 731.55
Podešeni hiperparametri sa zaostalim vrijednostima	3 856.97	5 415.62

Iz tablice 4.3 može se vidjeti da uvođenjem zaostalih vrijednosti smanjuje prosječni MAE svih modela za otprilike 80% te prosječni RMSE za oko 75% u odnosu na konfiguracije bez zaostalih vrijednosti. Dodatna optimizacija hiperparametara zatim donosi još oko 20% dodatnog smanjenja MAE i približno 20% smanjenja RMSE u odnosu na već proširene modele sa zaostalim vrijednostima.

Iz tablice 4.2 vidljivo je nekoliko ključnih zaključaka. Najbolji ukupni rezultat postiže optimizirani XGBoost s MAE = 3 405.16 kW i RMSE = 4 879.57 kW, čime nadmašuje sve promatrane arhitekture dubokog učenja na ovom skupu podataka. Za usporedbu, medijalna satna potrošnja energije iznosi 194 989.38 kW, pa prosječna apsolutna pogreška XGBoost modela odgovara relativnoj pogrešci od približno 1.75%. Drugi najbolji model je optimizirani GRU s MAE-om 3 680.79 kW, odnosno relativnom pogreškom oko 1.89%, što je približno 8% veća pogreška u odnosu na XGBoost. Razlika je relativno mala u apsolutnom smislu, ali je dovoljno izražena da XGBoost možemo označiti kao najpouzdaniji model za ovaj konkretan skup podataka.

Zanimljivo je da se, unatoč većoj izražajnoj snazi dubokih modela, najbolji rezultat postiže relativno klasičnim *Gradient Boosting* pristupom (XGBoost) nad zaostalim vrijednostima. Jedan od razloga vjerojatno leži u veličini skupa podataka i razini šuma u mjerenjima. Boosting modeli vrlo dobro iskorištavaju strukturirane tablične značajke, kao što su zaostale vrijednosti i meteorološke varijable, te su manje skloni preučanju na manjem broju uzoraka. S druge strane, sekvencijalni modeli poput LSTM-a, GRU-a, TCN-a i Transformer-a imaju znatno veći broj parametara i zahtijevaju više podataka i pažljivije regularizacijske strategije kako bi u potpunosti iskoristili svoj potencijal. U ovom radu njihov je učinak ipak vrlo dobar – primjerice, optimizirani GRU i Transformer postižu relativne pogreške ispod 2% – ali nešto lošiji od XGBoost-a.

5. ZAKLJUČAK

U ovom radu analiziran je problem predviđanja potrošnje električne energije primjenom metoda strojnog i dubokog učenja nad vremenskim nizovima. Na temelju skupa podataka koji uključuje povijesne vrijednosti potrošnje i meteorološke varijable provedena je eksplorativna analiza podataka, inženjerstvo značajki te izgradnja i evaluacija više različitih modela.

Rezultati eksperimentalne analize pokazali su da uključivanje zaostalih vrijednosti ima ključnu ulogu u uspješnosti modela, smanjuje prosječni MAE svih modela za otprilike 80% te prosječni RMSE za oko 75% u odnosu na konfiguracije bez zaostalih vrijednosti. Dodatno, optimizacija hiperparametara dovodi do dodatnog smanjenja pogreške, iako u manjoj mjeri u odnosu na utjecaj samih značajki.

U usporedbi različitih modela, najbolji rezultat ostvario je XGBoost model s podešenim hiperparametrima i uključenim zaostalim vrijednostima, koji je postigao najniže vrijednosti $MAE = 3\,405.16\text{ kW}$ i $RMSE = 4\,879.57\text{ kW}$. Time je pokazano da modeli temeljeni na gradijentnom pojačavanju mogu biti vrlo učinkoviti za predviđanje vremenskih nizova kada se koriste prikladno konstruirane značajke. Iako su modeli dubokog učenja (LSTM, GRU, TCN i Transformer) pokazali dobru sposobnost modeliranja vremenskih ovisnosti, njihova prednost nije u potpunosti došla do izražaja na ovom skupu podataka, što se može pripisati ograničenoj veličini skupa (8736 satnih zapisa) i složenosti modela.

Zaključno, rad potvrđuje da je za uspješno predviđanje potrošnje električne energije presudna kombinacija kvalitetne pripreme podataka i odgovarajućeg izbora modela. Kao moguća unaprjeđenja budućeg rada ističe se korištenje većih i raznolikijih skupova podataka, uključivanje dodatnih relevantnih značajki (npr. praznici, posebni događaji), primjena naprednijih arhitektura specijaliziranih za vremenske nizove te razvoj hibridnih modela koji kombiniraju prednosti klasičnih i dubokih metoda.

LITERATURA

- [1] Fedesoriano. Electric power consumption. <https://www.kaggle.com/datasets/fedesoriano/electric-power-consumption/data/>, August 2022.
- [2] Leo Breiman and Random Forests. Machine learning, vol. 45. *The Netherlands: Kluwer Academic Publishers*, pages 5–32, 2001.
- [3] CFI Team. Random forest. <https://corporatefinanceinstitute.com/resources/data-science/random-forest/>.
- [4] Tianqi Chen and Carlos Guestrin. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining*, pages 785–794, 2016.
- [5] Yuanchao Wang, Z. Pan, J. Zheng, L. Qian, and Li Mingtao. A hybrid ensemble method for pulsar candidate classification. *Astrophysics and Space Science*, 364, 08 2019.
- [6] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997.
- [7] Sireesh Haang Limbu. Direct speech to speech translation using machine learning, 2020.
- [8] Kyunghyun Cho, Bart Van Merriënboer, Çağlar Gulçehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using rnn encoder–decoder for statistical machine translation. In *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)*, pages 1724–1734, 2014.
- [9] Shiva Nosouhian, Fereshteh Nosouhian, and Abbas Kazemi Khoshouei. A review of recurrent neural network architecture for sequence learning: Comparison between lstm and gru, 2021.
- [10] Shaojie Bai, J Zico Kolter, and Vladlen Koltun. An empirical evaluation of generic convolutional and recurrent networks for sequence modeling. *arXiv preprint arXiv:1803.01271*, 2018.
- [11] Francesco Lässig. Temporal convolutional networks and forecasting. <https://unit8.com/resources/temporal-convolutional-networks-and-forecasting/>.
- [12] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- [13] Sivylla Paraskevopoulou. How to design transformer model for time-series forecasting. <https://blogs.mathworks.com/deep-learning/2024/11/12/how-to-design-transformer-model-for-time-series-forecasting/>.

PRILOG

Github repozitorij koji sadrži programski kod: [AndrejDamašek/SeminarskiRad_prilog](#)¹.

Skup podataka korišten za seminar preuzet s Kaggle: [AndrejDamašek/SeminarskiRad_prilog](#)².

¹<https://github.com/andrejdamasek/Predvidanje-potrosnje-energije-pomocu-vremenskih-nizova>

²<https://www.kaggle.com/datasets/fedesoriano/electric-power-consumption/data/>